

CLASSIFICAÇÃO COM KNN

Dataset: Phishing Websites - UCI Machine Learning Repository

OBJETIVO

Desenvolver um pipeline completo de classificação utilizando o algoritmo KNN para detectar websites phishing.

- Explorar o dataset
- Tratar inconsistências
- Aplicar normalizações
- Treinar modelos KNN
- Avaliar diferentes valores de k
- Aplicar validação cruzada
- Utilizar Grid Search
- Comparar resultados

PHISHING WEBSITES — UCI REPOSITORY

Características:

- 11.055 registros
- 30 atributos
- Classificação binária
- Sem valores ausentes

Classes:

- 1 → Website legítimo
- -1 → Website phishing

PIPELINE DESENVOLVIDO

- Carregamento do dataset
- Exploração dos dados
- Tratamento de inconsistências
- Análise de correlação
- Normalização
- Treinamento KNN
- Avaliação de diferentes valores de k
- Validação cruzada
- Grid Search
- Comparação dos resultados

EXPLORAÇÃO DO DATASET

Análises Realizadas:

- Quantidade de linhas e colunas
- Tipos dos atributos
- Distribuição das classes
- Valores únicos
- Estatísticas gerais

Resultado:

O dataset apresentou:

- boa organização
- ausência de valores nulos
- atributos numéricos adequados para KNN

TRATAMENTO DOS DADOS

Verificações Realizadas:

- Valores ausentes
- Dados inconsistentes
- Dados duplicados

Resultados:

- Não havia valores nulos
- Foram encontradas 5.270 linhas duplicadas

Ação Realizada:

Remoção das duplicatas para reduzir vieses.

Resultado Final:

- 5.849 registros restantes

ANÁLISE DE CORRELAÇÃO

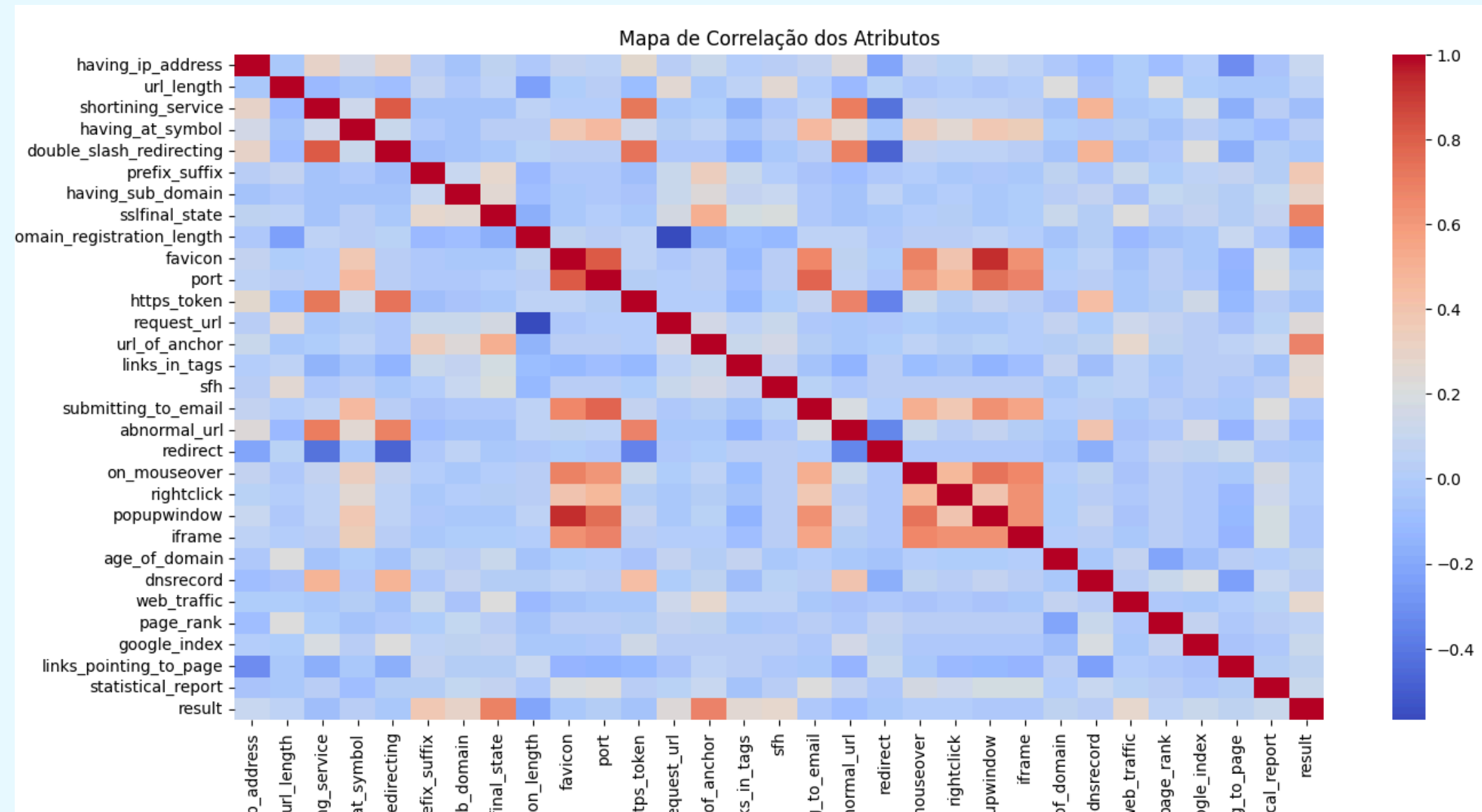
Objetivo:

Identificar os atributos mais relevantes para classificação.

Principais Atributos Correlacionados

Atributo	Correlação
sslfinal_state	0.693
url_of_anchor	0.679
prefix_suffix	0.381

CORRELAÇÃO ENTRE OS ATRIBUTOS



O heatmap mostrou forte relação entre alguns atributos ligados à segurança da URL e a variável alvo.

NORMALIZAÇÃO DOS DADOS

Por que normalizar?

- O KNN utiliza distância entre pontos.

Sem normalização:

- atributos maiores dominam o cálculo da distância.

Técnicas utilizadas:

- Min-Max
- Z-Score
- Log

TREINAMENTO DO KNN

Configuração Inicial

Algoritmo:

- KNN
- $k = 5$

Divisão:

- 80% treino
- 20% teste

Resultado:

- Acurácia = 92,31%

MATRIZ DE CONFUSÃO

Resultado do Modelo

	Predito -1	Predito 1
Real -1	580	40
Real 1	50	500

O modelo apresentou boa capacidade de classificação em ambas as classes.

CLASSIFICATION REPORT

Classe	Precision	Recall	F1-score
-1	0.92	0.94	0.93
1	0.93	0.91	0.92

O modelo apresentou equilíbrio entre:

- precisão;
- recall;
- F1-score.

INFLUÊNCIA DO PARÂMETRO K

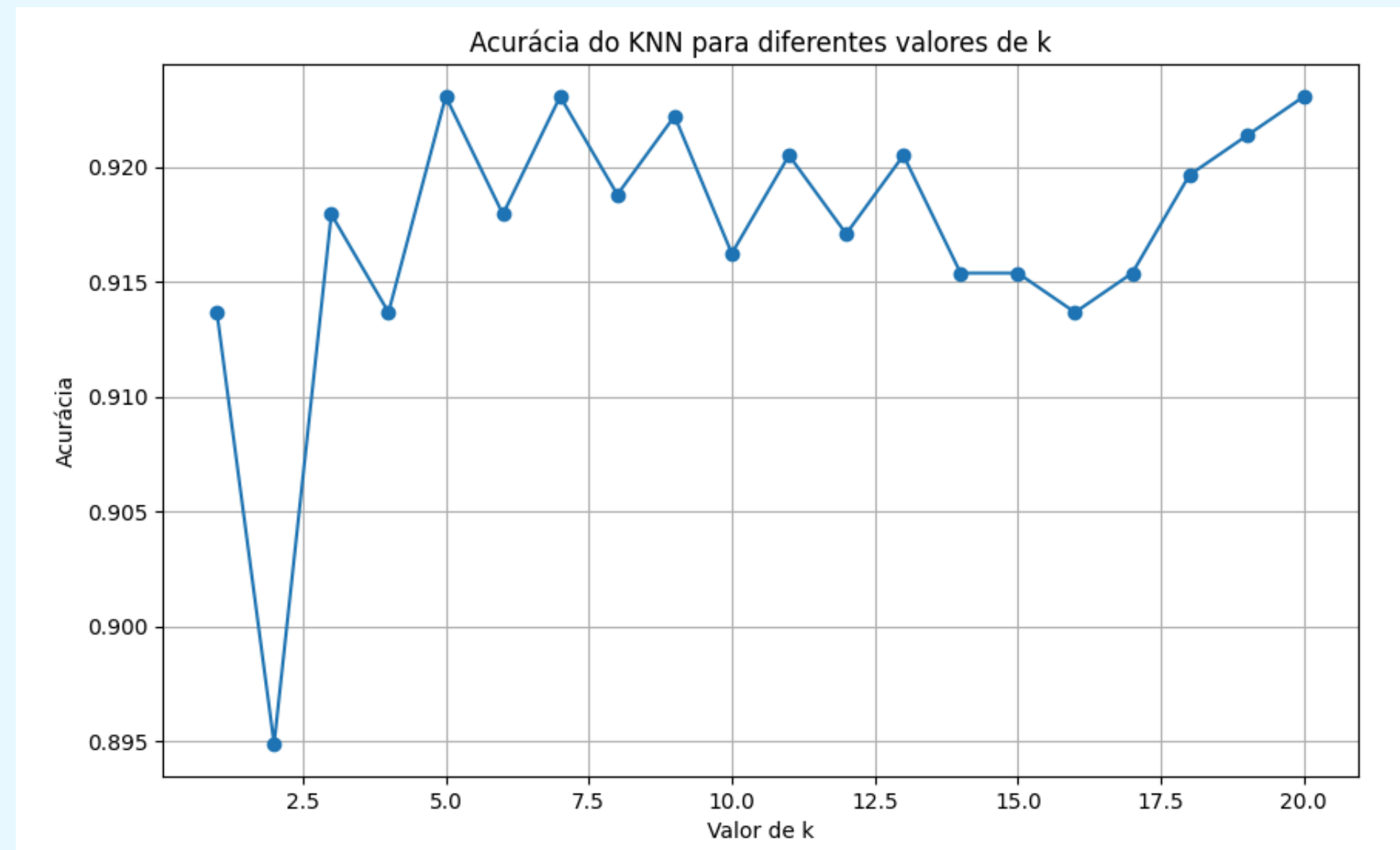
Objetivo:

- Analisar o impacto do valor de k no desempenho do KNN.

Valores testados:

- $k = 1$ até $k = 20$

GRÁFICO K VS ACURÁCIA



Melhor resultado visual:

- Melhor k = 5
- Acurácia = 0.9231

VALIDAÇÃO CRUZADA

Objetivo:

- Avaliar a capacidade de generalização do modelo.

Configuração:

- Cross Validation
- 5 folds

Melhor resultado:

- $k = 7$
- Acurácia = 0.9077

GRID SEARCH

Objetivo:

- Encontrar automaticamente os melhores hiperparâmetros.

Parâmetros testados:

- n_neighbors
- metric
- weights

Melhor configuração:

- metric = manhattan
- n_neighbors = 6
- weights = distance

Melhor acurácia:

- 93,01%

COMPARAÇÃO DOS RESULTADOS

Método	Resultado
Análise Visual	$k = 5$
Cross Validation	$k = 7$
Grid Search	$k = 6$

O Grid Search apresentou o melhor desempenho.

COMPARAÇÃO DAS NORMALIZAÇÕES

Normalização	Acurácia
Min-Max	0.9065
Z-Score	0.9104
Log	0.9101

Melhor técnica = Z-Score

CONCLUSÕES

Principais conclusões:

- O KNN apresentou excelente desempenho
- A normalização influenciou diretamente os resultados
- Valores intermediários de k foram mais estáveis
- Grid Search melhorou os resultados
- A validação cruzada confirmou boa generalização

PRÓXIMOS PASSOS

Melhorias futuras:

- Testar Random Forest e SVM
- Aplicar seleção automática de atributos
- Trabalhar com datasets maiores
- Criar sistema em tempo real
- Aplicar técnicas de balanceamento

**OBRIGADO
PELA ATENÇÃO!**